Using Micro Python for IoT

ESP32 + DHT22 Temperature and Humidity Monitoring using MQTT

Group Members:

1) Ryan Gicheru- 162484
2) Mark Talamson-154311
3) Zarian Achieng'-153689
4) Isaac Kariuki-151442
5) Peter Kimori-169613
6) Nigel Thoya-169980

Wokwi Simulation: https://wokwi.com/projects/466736337106310145

GitHub Repository: https://github.com/JUST-ZARI/IOT-micropython-lab

1. Introduction

In this laboratory exercise, an ESP32 microcontroller and a DHT22 temperature and humidity sensor were used to develop a complete IoT monitoring system. The DHT22 sensor was responsible for measuring environmental conditions, while the ESP32 processed the sensor readings and transmitted them over a wireless network. Communication between devices was achieved using the MQTT protocol, a lightweight messaging protocol commonly used in IoT applications due to its efficiency and low bandwidth requirements.

A Python-based subscriber application was developed to receive the published sensor readings. The received data was then stored in a SQLite database, providing a persistent record of environmental measurements. This lab demonstrated the integration of sensing, communication, data processing, and storage components within a single IoT system while providing practical experience with Micro Python, MQTT, and database technologies.

2. Objectives

i. Flash Micro Python firmware onto the ESP32 using esptool from the command line.
ii. Interface the DHT22 temperature and humidity sensor with the ESP32 and obtain sensor readings.
iii. Write Micro Python code to read temperature and humidity data from the DHT22 sensor.

iv. Configure the ESP32 to connect to a Wi-Fi network.
v. Run a local Mosquitto MQTT broker and establish MQTT communication between devices.
vi. Publish sensor readings as structured JSON payloads to an MQTT topic using the ESP32.
vii. Write a Python subscriber application using paho-mqtt to subscribe to the MQTT topic and process incoming JSON messages.
viii. Store received sensor readings into a SQLite database together with timestamps.
ix. Verify successful end-to-end IoT communication from sensor data acquisition to database storage.
x. Identify and resolve issues encountered during sensor setup, Wi-Fi connectivity, MQTT communication, and database storage.

3.Hardware and Software Requirements

3.1 Hardware Components

Component	**Purpose**
USB Data Cable	Power and connection to PC
10 kΩ Resistor	Pull-up on DHT22 data line to ensure signal stability
Jumper Wires	VCC, GND, and DATA connections(Component Connections)
Breadboard	Circuit Assembly
DHT22 Sensor	Measures ambient temperature and humidity
ESP32 Development Board	Main microcontroller

3.2 Software Components

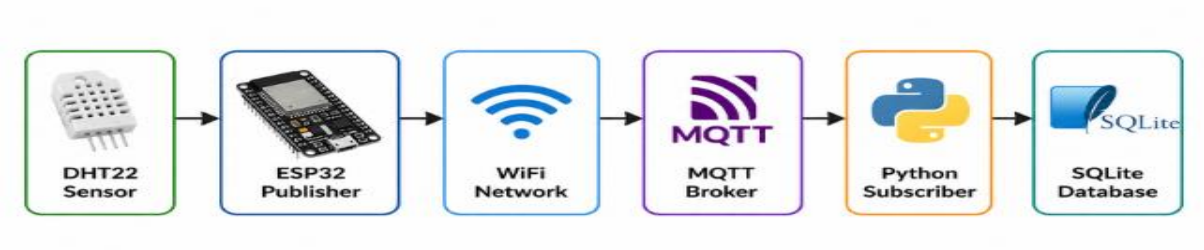
Component	**Purpose**
Micro Python	ESP32 programming
Thonny IDE	Code development
Paho MQTT	Python MQTT library

Mosquitto Broker	MQTT communication
SQLite	Database storage for sensor readings
Python 3.x	Subscriber implementation
esptool	Flash Micro Python firmware to the ESP32

4. System Architecture

The DHT22 sensor measures environmental conditions. The ESP32 reads these values and publishes them as JSON messages to the MQTT broker. The subscriber receives the messages and stores them in a SQLite database.

Figure 1: Overall System Architecture



4.1 Project Implementation Stages

i. Software Installation

The required software tools, including Python 3.x, Thonny IDE, Micro Python, Mosquitto MQTT broker, SQLite, and paho-mqtt, were installed and configured for the project.

ii. ESP32 Firmware Preparation

The ESP32 flash memory was erased using esptool to prepare the board for Micro Python installation.

iii. Micro Python Firmware Installation

Micro Python firmware was flashed onto the ESP32 and tested through Thonny IDE to confirm successful communication with the board.

iv. Hardware Assembly

The DHT22 sensor was connected to the ESP32 using GPIO 4, with a 10 kΩ pull-up resistor added to ensure reliable data transmission.

v. Wi-Fi and MQTT Configuration

The ESP32 was configured to connect to the Wi-Fi network and communicate with the MQTT broker.

vi. Sensor Data Acquisition and Publishing

Temperature and humidity readings were obtained from the DHT22 sensor and published to an MQTT topic as JSON messages.

vii. MQTT Subscription and Data Storage

A Python subscriber application received the MQTT messages and stored the sensor readings in a SQLite database.

viii. Testing and Verification

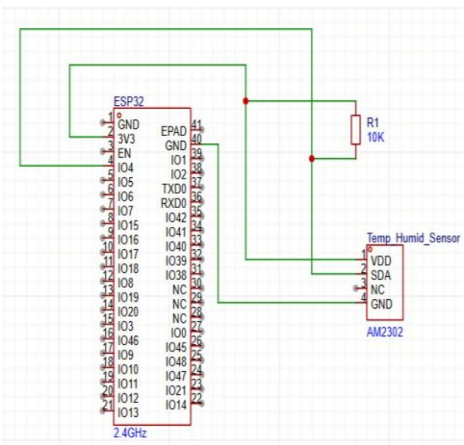
The complete system was tested to verify successful sensor data collection, MQTT communication, and database storage.

5. Implementation

5.1 Circuit Design

The DHT22 sensor was connected to GPIO 4 of the ESP32 together with a pull-up resistor.

Figure 2: Simulation Circuit



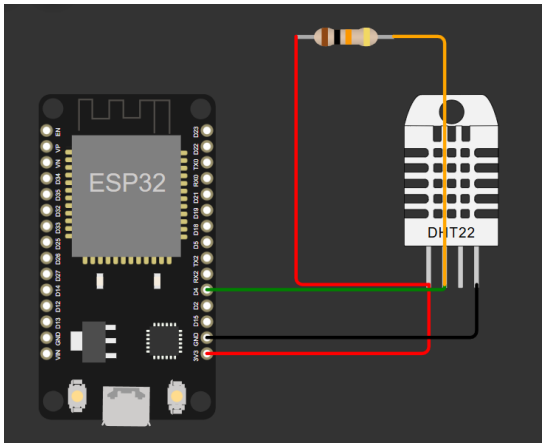
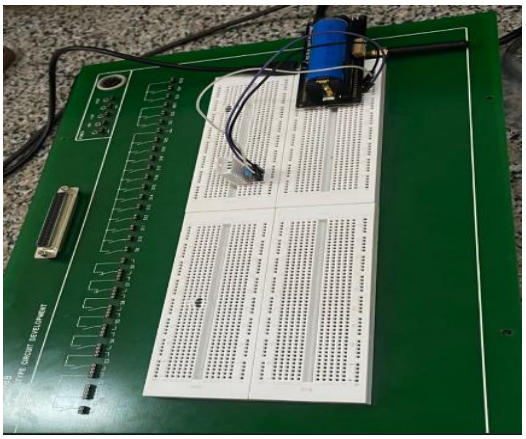


Figure 3: Physical Hardware Setup



The DHT22 sensor was connected to the ESP32 using GPIO 4 as the data pin. The sensor's VCC and GND pins were connected to the ESP32 power rails, while a 10 kΩ pull-up resistor was placed between VCC and the data line to ensure stable communication.

5.2 Wi-Fi Configuration

The ESP32 was configured to connect to a wireless network before transmitting sensor data. The Micro Python application used the network module to initialize the station interface and establish a connection to the configured Wi-Fi access point. Once connected, the ESP32 obtained an IP address, enabling communication with the MQTT broker.

Figure 4: Wi-Fi Connection Configuration

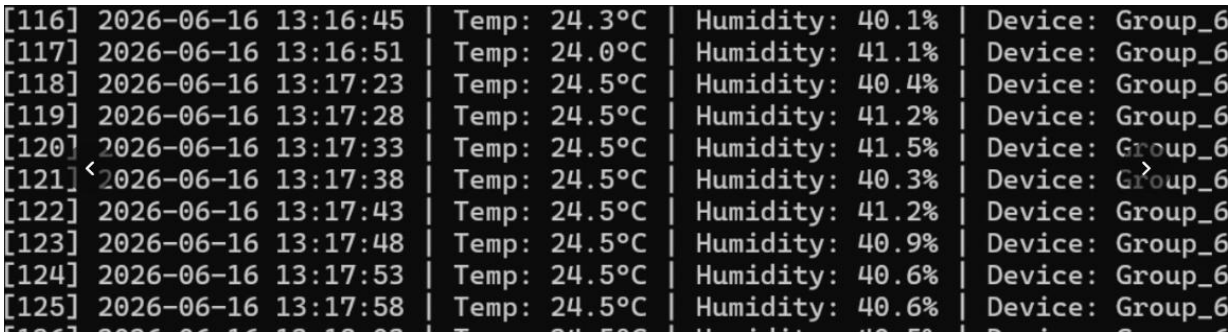
```
# Configuration
WIFI_SSID     = "Talamson"   # Wokwi's built-in WiFi AP
WIFI_PASSWORD = "0792314330"               # No password in Wokwi
```

```
def connect_wifi():
    wlan = network.WLAN(network.STA_IF)

    # Full reset of WiFi radio
    wlan.active(False)
    time.sleep(1)
    wlan.active(True)
    time.sleep(1)

    if not wlan.isconnected():
        print(f"[WiFi] Connecting to '{WIFI_SSID}' ...")
        wlan.connect(WIFI_SSID, WIFI_PASSWORD)
```

5.3 Sensor Reading and MQTT Publishing

The ESP32 was programmed using Micro Python to read temperature and humidity values from the DHT22 sensor every five seconds. Each reading was formatted as a JSON message and published to the MQTT.

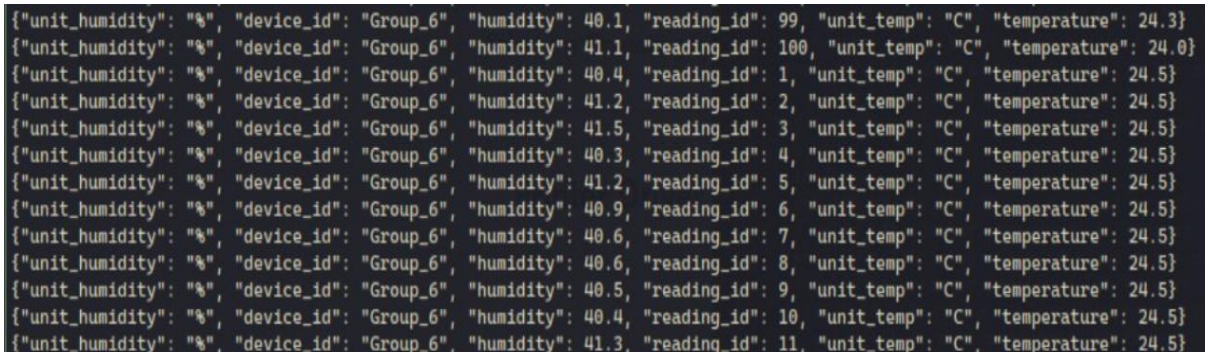
Figure 5: REPL Output Showing Live Sensor Readings



5.4 MQTT Subscription

A Python subscriber application using the paho-mqtt library was developed. The subscriber received incoming JSON messages and displayed them in the terminal.

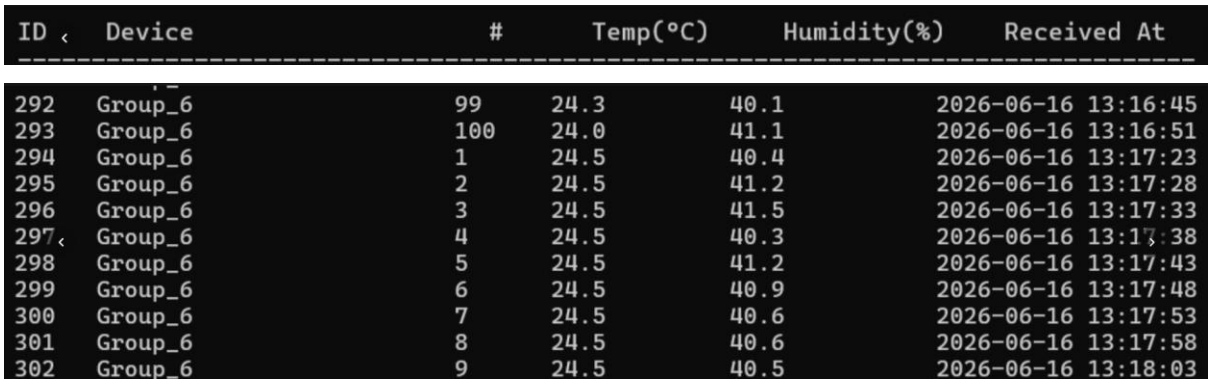
Figure 6: Mosquitto Subscriber Receiving Messages



5.5: Database Storage

The subscriber stored all received sensor readings in a SQLite database together with timestamps.

Figure 7: SQLite Database Query Results



ID	Device	#	Temp(°C)	Humidity(%)	Received At
292	Group_6	99	24.3	40.1	2026-06-16 13:16:45
293	Group_6	100	24.0	41.1	2026-06-16 13:16:51
294	Group_6	1	24.5	40.4	2026-06-16 13:17:23
295	Group_6	2	24.5	41.2	2026-06-16 13:17:28
296	Group_6	3	24.5	41.5	2026-06-16 13:17:33
297	Group_6	4	24.5	40.3	2026-06-16 13:17:38
298	Group_6	5	24.5	41.2	2026-06-16 13:17:43
299	Group_6	6	24.5	40.9	2026-06-16 13:17:48
300	Group_6	7	24.5	40.6	2026-06-16 13:17:53
301	Group_6	8	24.5	40.6	2026-06-16 13:17:58
302	Group_6	9	24.5	40.5	2026-06-16 13:18:03

6. Results

During testing, the ESP32 successfully connected to the Wi-Fi network and continuously published temperature and humidity readings every five seconds. The MQTT subscriber received more than ten JSON messages without loss, and all received records were successfully stored in the SQLite database with timestamps. The results confirmed successful end-to-end communication between the sensor, ESP32, MQTT broker, subscriber application, and database.

7. Challenges Encountered and Solutions

1. **Challenge 1: Wi-Fi Connectivity:** The ESP32 initially failed to connect to the network due to incorrect credentials.

   Solution : The Wi-Fi configuration was verified, and connection timeout handling was added.

2. **Challenge 2: Sensor Read Errors :**The DHT22 occasionally produced OSError exceptions.

   Solution: Exception handling was implemented using:

   except OSError as e:

   which prevented program termination.

3. **Challenge 3: Database Verification:** Initially, it was difficult to confirm whether readings were being stored correctly.

Solution: The query functionality in subscriber.py was used to retrieve all stored records and verify successful insertion.

8. Conclusion

This laboratory exercise successfully demonstrated the implementation of an Internet of Things (IoT) monitoring system using an ESP32 microcontroller, a DHT22 temperature and humidity sensor, MQTT messaging, and SQLite database storage. The project provided practical experience in configuring embedded devices, implementing wireless communication, integrating MQTT-based data exchange, and managing sensor data persistence.

The successful operation of the system confirmed the effectiveness of using Micro Python and MQTT for lightweight IoT applications. Through this exercise, valuable skills were gained in hardware interfacing, firmware deployment, network communication, troubleshooting, and database integration.